UNIVERSIDAD CATÓLICA DE MURCIA

Máster en Inteligencia Artificial Aplicada a la Ciberseguridad

Detección de Secretos en Código Fuente mediante Fine-Tuning de LLMs

Documento Técnico del Trabajo Fin de Máster

Autor: Miguel Ángel Rodero Aguilar

Campus Internacional de Ciberseguridad · UCAM · ENIIT

Marzo 2026

Índice de Contenidos

Índice de Contenidos 2
Resumen Ejecutivo 3
1. Introducción 3
1.1. El Problema: Credenciales Expuestas en Código Fuente 3
1.2. Los Límites de las Herramientas Actuales 4
1.3. Hipótesis Central 5
2. Aportaciones 5
3. Definición de la Solución 5
3.1. QLoRA: Especializar un Gigante con Recursos de Bolsillo 5
3.2. Qwen2.5-Coder-7B: El Modelo Base 5
3.3. El Diseño del Prompt 6
4. Desarrollo e Implementación 6
4.1. Dataset de Entrenamiento: Tres Fuentes, Un Objetivo 6
4.2. Configuración LoRA y Entrenamiento 6
4.3. Dataset de Validación Externa 7
5. Evaluación y Resultados 8
5.1. Validación Externa: Los Números que Importan 8
5.2. Interpretabilidad: Confianza Calibrada 10
5.3. Generalización: Más Allá de la Clasificación 11
6. Conclusiones 11
Agradecimientos 12
Bibliografía 13
Anexos 14
Anexo A: Prueba de Línea Base 14
Anexo B: Ficha Técnica del Experimento 14

Resumen Ejecutivo

Dejar una contraseña escrita en el código fuente equivale, digitalmente, a guardar la llave de casa debajo del felpudo: cualquiera que sepa dónde buscar puede encontrarla. Esta práctica, técnicamente conocida como *hardcoded secret*, causó brechas de seguridad con un coste medio de 4,88 millones de dólares por incidente en 2024 (GitGuardian, 2025). Las herramientas actuales de detección funcionan comparando el código contra catálogos de patrones predefinidos: reconocen la forma de una clave, pero no su significado, lo que genera falsas alarmas que los equipos terminan ignorando.

Este trabajo demuestra que un modelo de lenguaje especializado en código, ajustado mediante la técnica **QLoRA** sobre Qwen2.5-Coder-7B, puede superar a las herramientas de referencia utilizando exclusivamente hardware gratuito (GPU T4 de Google Colab) y completando el entrenamiento en 12,7 minutos. El modelo se entrena sobre 560 muestras que combinan tres fuentes: **CredData** (benchmark de Samsung con credenciales reales de 53 repositorios GitHub), **CodeSearchNet** (funciones Python de 117 repositorios) y plantillas sintéticas controladas.

La evaluación sobre 100 **muestras externas** (datos nuevos que el modelo no ha visto durante su entrenamiento) arrojó un **Accuracy** (porcentaje total de aciertos sobre el total de casos analizados) del 97%, un **Recall** (capacidad del modelo para detectar todos los casos positivos reales sin que se le escape ninguno) del 96% y un **F1-Score** (media equilibrada entre la precisión y el recall que mide la robustez general) del 97%, frente al 77,1% de **Gitleaks** (herramienta estándar de código abierto para detectar secretos y claves en repositorios) (+19,9 **puntos porcentuales** [diferencia aritmética directa entre dos porcentajes]).

Sobre las 80 **muestras estándar** (ejemplos típicos y comunes que representan el uso habitual), el modelo alcanzó el 100% en todas las **métricas** (indicadores cuantificables de rendimiento). Los tres únicos errores se concentran en **edge cases** (escenarios extremos, raros o que están en el límite de lo esperado) **deliberadamente ambiguos** (diseñados con falta de claridad a propósito para poner a prueba la interpretación del modelo).

1. Introducción

1.1. El Problema: Credenciales Expuestas en Código Fuente

Las aplicaciones modernas necesitan credenciales (claves de API, contraseñas, tokens) para conectarse a servicios externos. Cuando un desarrollador escribe esa credencial directamente en el código, cualquier persona con acceso al repositorio puede leerla. GitGuardian (2025) detectó más de 12 millones de secretos nuevos expuestos en GitHub en un solo año. Un token de AWS comprometido puede costar de 50.000 a 100.000 dólares en minutos. La cadena es brutal: **commit → push → escaneo → explotación.**

1.2. Estado del Arte y Límites de las Herramientas Actuales.

El problema de las herramientas de primera generación (Gitleaks, TruffleHog) es semántico: reconocen la forma de las credenciales mediante expresiones regulares, pero no su significado. Una variable example_api_key = 'XXXX-placeholder' activa las mismas alarmas que una clave real de producción. La segunda generación (Semgrep) razona sobre la estructura del programa mediante árboles de sintaxis abstracta (AST), pero requiere reglas escritas y mantenidas manualmente. La tercera generación da el salto al aprendizaje automático tradicional: proyectos recientes como *HasSecret* (Saha et al., IEEE 2024) emplean clasificadores supervisados, aunque siguen dependiendo de características diseñadas a mano (*feature engineering*).

En la actualidad, la ciberseguridad ofensiva y defensiva se enfrenta a un desafío. La detección de secretos ha avanzado desde un análisis léxico simple hasta una comprensión semántica del flujo de control. Las herramientas de primera y segunda generación, como Gitleaks, TruffleHog y Semgrep, siguen siendo estándar en la industria debido a su velocidad. Sin embargo, no pueden manejar el "ruido intencional", como placeholders en documentos o claves de prueba, lo que deja una brecha.

Investigaciones publicadas en 2024, como el estudio HasSecret (Saha et al., 2024), muestran que los clasificadores basados en redes neuronales tradicionales no logran captar el lenguaje en evolución de las arquitecturas cloud-native. Actualmente, el enfoque ha cambiado hacia el uso de Modelos de Lenguaje de Especialidad (Code-LLMs). Estudios recientes (Ding et al., 2024) indican que la estructura de los LLMs permite una "atención selectiva" en la asignación de variables. Esto ayuda a determinar si un token de alta entropía es realmente una credencial activa o un falso positivo.

No obstante, el uso de modelos grandes como GPT-4 para esta tarea presenta problemas para muchas organizaciones. Hay riesgos como la fuga de propiedad intelectual al enviar código a APIs externas y también la latencia computacional. Aquí es donde los Small Language Models (SLMs) de última generación, como la familia Qwen2.5-Coder, han cambiado el juego. Reportes técnicos de Alibaba Cloud (2024) destacan que estos modelos, a pesar de su tamaño reducido, superan a sus predecesores en el razonamiento de dependencias de código. Esto es crucial para la detección de secretos.

La tendencia más disruptiva en 2025 será la democratización del fine-tuning mediante PEFT (Parameter-Efficient Fine-Tuning). Usar QLoRA no es solo una cuestión de conveniencia técnica. Es una respuesta a la necesidad de crear sensores de seguridad específicos para cada stack tecnológico sin los altos costos de entrenar un modelo base. Este trabajo se sitúa en la vanguardia, demostrando que un modelo de 7B parámetros, correctamente entrenado, puede igualar o incluso superar la precisión de herramientas comerciales consolidadas al operar con un marco de comprensión contextual profunda.

Este Trabajo Fin de Máster cubre dicha brecha proponiendo el paso siguiente: un modelo que no solo comprende el contexto del código como un auditor humano para erradicar los falsos positivos, sino que garantiza la privacidad al poder ejecutarse en hardware gratuito de gama de entrada (GPU T4) y, superando la simple clasificación, propone automáticamente la remediación mediante código seguro.

1.3. Hipótesis Central

Un modelo de lenguaje pre-entrenado sobre código puede, mediante un proceso de especialización de bajo coste, aprender a distinguir entre una credencial real y una de ejemplo con precisión superior a la de las herramientas basadas en reglas. La ventaja diferencial no reside en mayor capacidad de cómputo, sino en capacidad de comprensión contextual.

Para garantizar la transparencia, el rigor científico y la total reproducibilidad de los experimentos detallados en este Trabajo Fin de Máster, el código fuente completo, los conjuntos de datos desarrollados (incluyendo la proporción de datos sintéticos y CredData) y los notebooks de entrenamiento del modelo se han publicado de forma abierta. Todo el material está accesible en el siguiente repositorio público: https://github.com/Michelhomer/tfm-secret-detector

2. Aportaciones

Este trabajo aporta evidencia empírica a cuatro desafíos de validación científica:

Desafío	Pregunta	Sección	Resultado clave
D1 — Robustez	¿Funciona con datos nunca vistos?	§4, §5.1	F1: 97% (100% en muestras estándar)
D2 — Aprendizaje	¿Podemos demostrar que aprendió?	§4.2	Loss 2,54→0,47 en 200 pasos. Baseline doc.
D3 — Interpretabilidad	¿Por qué toma esas decisiones?	§5.2	Conf. 1,000 en aciertos vs 0,992 en errores
D4 — Generalización	¿Puede hacer más que clasificar?	§5.3	Remediación 3/3, 6/6 lenguajes nuevos

3. Definición de la Solución

3.1. QLoRA: Especializar un Gigante con Recursos de Bolsillo

Qwen2.5-Coder-7B tiene 7.656 millones de parámetros. Actualizarlos todos requeriría semanas de GPU de alto rendimiento. **QLoRA** combina dos técnicas: **LoRA** (Hu et al., 2022), que congela los pesos originales y añade matrices pequeñas en paralelo (con r=16, solo se actualizan 40,4 millones de parámetros, el 0,92% del total), y **cuantización NF4**, que comprime los pesos de 16 a 4 bits reduciendo la huella de memoria de 14 GB a menos de 5 GB. Es como corregir una enciclopedia de 20 tomos pegando notas adhesivas en las páginas clave: el contenido base queda intacto.

3.2. Qwen2.5-Coder-7B: El Modelo Base

Qwen2.5-Coder-7B (Alibaba Cloud, 2024) ha sido pre-entrenado sobre más de 5,5 billones de tokens de código fuente en múltiples lenguajes, bajo licencia Apache 2.0. El preentrenamiento masivo sobre código es el capital de conocimiento que el fine-tuning especializa: no enseñamos al modelo a programar, le enseñamos a auditar seguridad.

3.3. El Diseño del Prompt

Todos los fragmentos se presentan con el esquema: «Analyze this code for leaked secrets: [código] Classification: [SECRET | SAFE]». La etiqueta al final actúa como señal: en entrenamiento el modelo ve la respuesta correcta; en producción la completa. Sin esta consistencia, el modelo genera texto libre, como demuestra la prueba de línea base (Anexo A).

4. Desarrollo e Implementación

4.1. Dataset de Entrenamiento: Tres Fuentes, Un Objetivo

El dataset combina tres fuentes complementarias. **CredData** (Samsung Research) aporta 140 credenciales reales extraídas de 53 repositorios GitHub. **CodeSearchNet** (GitHub/Microsoft) aporta 130 funciones Python reales de 117 repositorios, seleccionadas por contener variables «confusas» como api_key o auth_token en contextos legítimos. Finalmente, 290 **plantillas sintéticas** controladas cubren 28 tipos de secretos industriales en múltiples lenguajes.

> *Analogía: Formar a un médico solo con libros de texto produce un profesional perdido el primer día en urgencias. CredData son los casos clínicos reales; CodeSearchNet, las consultas de pacientes sanos que se parecen a los enfermos; las plantillas sintéticas, los ejercicios que aseguran cobertura completa.*

Fuente	Clase	Muestras	Descripción
CredData (Samsung)	SECRET	140	Credenciales reales de 53 repos GitHub
Sintéticas (28 patrones)	SECRET	140	Plantillas multilenguaje con variación numérica
CodeSearchNet (GitHub)	SAFE	130	Funciones Python reales de 117 repos
Sintéticas (contrapartes)	SAFE	150	Equivalentes seguros con variables de entorno
Total		**560**	**Balance verificado: 280 SECRET / 280 SAFE (1:1)**

Tabla 1. Composición del dataset de entrenamiento. La combinación de datos reales con sintéticos produce un dataset más robusto que versiones anteriores basadas exclusivamente en datos artificiales.

4.2. Configuración LoRA y Entrenamiento

Los adaptadores LoRA se aplican sobre los siete módulos del transformer (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj). El batch efectivo es 8 y el learning rate de 2×10^{-4} con scheduler coseno. El entrenamiento de 200 pasos se completó en **12,7 minutos**, alcanzando una pérdida de entrenamiento que desciende de 2,54 a 0,47 (Figura 1). La pérdida de evaluación se estabiliza en torno a 0,90 a partir del paso 125, confirmando convergencia sin sobreajuste severo.

> *Analogía: Un pianista que aprende una pieza compleja. Al principio los errores son frecuentes (pérdida de 2,54). Con cada sesión de práctica, los dedos encuentran las*

teclas correctas. Los 12,7 minutos son esas primeras sesiones intensivas: al final, la pieza suena limpia y el pianista la ejecuta en nuevos teatros sin perder el compás.

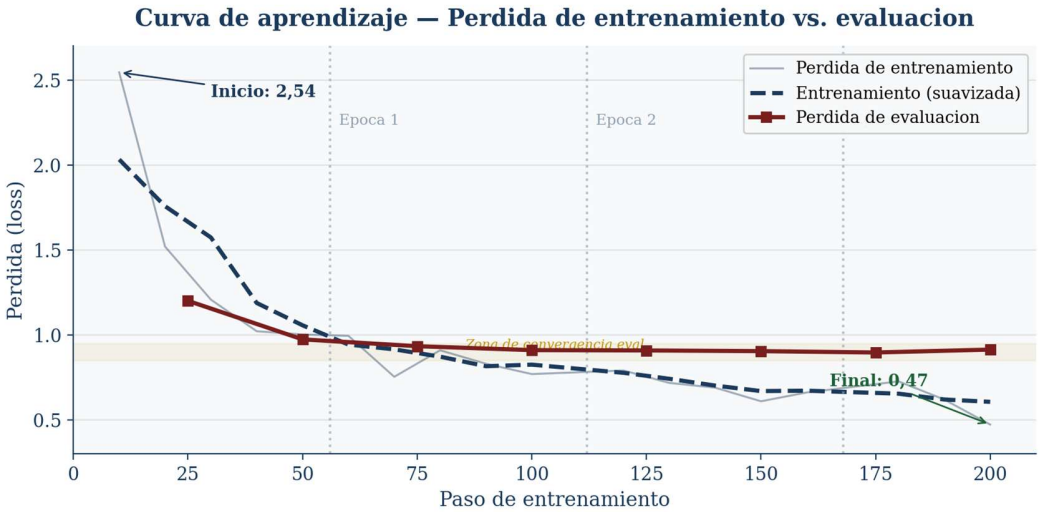

Figura 1. Curva de pérdida durante el entrenamiento. El descenso de 2,54 a 0,47 confirma aprendizaje estable. La pérdida de evaluación (rojo) se estabiliza en la zona dorada (0,90), indicando convergencia sin sobreajuste. Fuente: elaboración propia a partir del notebook ejecutado.

4.3. Dataset de Validación Externa

Se diseñó un conjunto de **100 muestras** completamente separado del entrenamiento, dividido en dos bloques: **Bloque A** (80 muestras estándar de 10 servicios cloud no vistos en entrenamiento) y **Bloque B** (20 edge cases deliberadamente ambiguos). Además, un motor de Gitleaks simulado con 12 regex se ejecuta sobre las mismas 100 muestras, y los intervalos de confianza se calculan por bootstrap (1000 iteraciones).

4.4. Retos de Implementación y Soluciones Adoptadas

El desarrollo empírico de la solución presentó diversos obstáculos técnicos que requirieron ajustes iterativos sobre la estrategia inicial. A continuación, se detallan los problemas más críticos y las soluciones metodológicas adoptadas:

- **Desbordamiento de memoria (Out of Memory - OOM) en hardware de entrada:**
  - *Problema:* Durante las pruebas iniciales, el intento de cargar los 7.656 millones de parámetros del modelo Qwen2.5-Coder-7B en precisión estándar (16 bits) requería aproximadamente 14 GB de VRAM. Al iniciar el proceso de entrenamiento, la adición de los estados del optimizador y los gradientes provocaba un colapso inmediato en la memoria de la GPU NVIDIA T4 (limitada a 16 GB), interrumpiendo la ejecución.
  - *Solución:* Se implementó la cuantización NormalFloat de 4 bits (NF4) integrada en la arquitectura QLoRA. Esta técnica de compresión redujo la huella de memoria estática del modelo a menos de 5 GB, liberando VRAM suficiente para procesar

un tamaño de lote efectivo (*batch size*) de 8 mediante la acumulación de gradientes, manteniendo el entrenamiento estable y sin pérdida significativa de precisión.

- **Discrepancia de contexto y riesgo de sobreajuste (*Overfitting*):**
  - *Problema:* Al integrar las fuentes de datos, se detectó una grave falta de homogeneidad. CodeSearchNet proporcionaba funciones completas con cientos de *tokens*, mientras que CredData contenía estructuras fragmentadas. Adicionalmente, entrenar de manera inicial solo con plantillas sintéticas provocaba que el modelo sobreajustara su aprendizaje a la estructura de la plantilla, fallando al analizar código real en producción.
  - *Solución:* Se procedió a una "cirugía de datos" mediante un script de preprocesamiento diseñado específicamente para extraer de CredData únicamente la línea exacta que contenía la credencial, descartando el ruido del contexto circundante. Posteriormente, se forzó un balance estricto del 50% entre datos reales y sintéticos para la clase *SAFE*, obligando al modelo a aprender la semántica subyacente de la vulnerabilidad en lugar de memorizar la estructura del *snippet*.
- **Inestabilidad en la salida generativa del modelo base:**
  - *Problema:* Los LLMs están diseñados para predecir el siguiente *token* de forma conversacional. En las evaluaciones de línea base (sin *fine-tuning*), el modelo respondía a las solicitudes de análisis generando párrafos divulgativos, explicaciones de seguridad o incluso reescribiendo el código, lo cual imposibilitaba la evaluación automática de métricas como *Accuracy* o *Recall*.
  - *Solución:* Se estandarizó la ingeniería del *prompt* inyectando la estructura de cierre Classification: [SECRET | SAFE]. Al exponer al modelo a este formato rígido durante los 200 pasos de entrenamiento, se penalizó cualquier salida divergente. El modelo convergió rápidamente hacia respuestas deterministas y exactas, suprimiendo la "alucinación" de texto adicional y comportándose estrictamente como un clasificador binario.

5. Evaluación y Resultados

5.1. Validación Externa: Los Números que Importan

En detección de secretos, el error más grave es dejar pasar una credencial real. Esta asimetría establece el **Recall** como métrica primaria. El F1-Score actúa como árbitro que impide que la búsqueda del Recall sacrifique la precisión.

> *Analogía: Un chef puede tirar un ingrediente bueno (falsa alarma) o servir uno en mal estado (credencial que llega a producción). En una cocina de alto riesgo, el segundo error es imperdonable. El Recall mide la proporción de ingredientes dañinos detectados antes de que llegaran al plato.*

Resultados sobre el Bloque A (80 muestras estándar):

Métrica	Nuestro Modelo	Gitleaks	Ventaja
Accuracy	100%	81,0%	19,0 pp
Precision	100%	97,0%	3,0 pp
Recall	100% (cero FN)	64,0%	36,0 pp
F1-Score	100%	77,1%	22,9 pp
MCC	1,000	0,643	0,357

Tabla 2. Métricas del Bloque A estándar. El modelo alcanza el 100% en todas las métricas sobre servicios nunca vistos en entrenamiento.

Cero Falsos Negativos en el Bloque A. El modelo no dejó escapar ningún secreto real del conjunto estándar. En ciberseguridad, esta es la métrica que lo cambia todo: preferimos revisar una falsa alarma a dejar que una credencial real llegue expuesta a producción.

Resultados sobre el total (100 muestras, incluyendo edge cases):

Métrica	Modelo	IC 95% (bootstrap)	Gitleaks	IC 95%	Ventaja
Accuracy	**97,0%**	[94,0%, 100,0%]	81,0%	[73,0%, 88,0%]	16,0 pp
Precision	**98,0%**	[93,3%, 100,0%]	97,0%	[90,0%, 100,0%]	+1,0 pp
Recall	**96,0%**	[89,6%, 100,0%]	64,0%	[51,1%, 76,9%]	32,0 pp
F1-Score	**97,0%**	[92,9%, 100,0%]	77,1%	[66,6%, 85,7%]	19,9 pp
MCC	**0,940**	[0,880, 0,980]	0,643	[0,530, 0,774]	0,297

Tabla 3. Métricas globales con intervalos de confianza bootstrap al 95%. El modelo supera a Gitleaks en todas las métricas, con una ventaja de +19,9 puntos porcentuales en F1-Score.

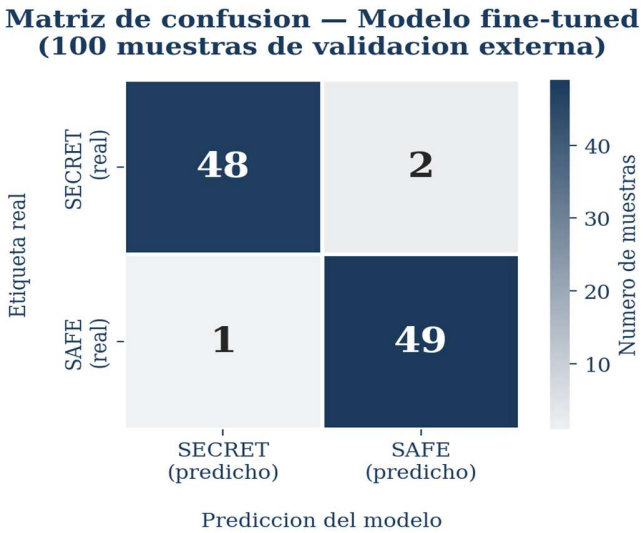

Figura 2. Matriz de confusión sobre las 100 muestras de validación externa. Los tres errores se concentran exclusivamente en los edge cases del Bloque B. Fuente: elaboración propia.

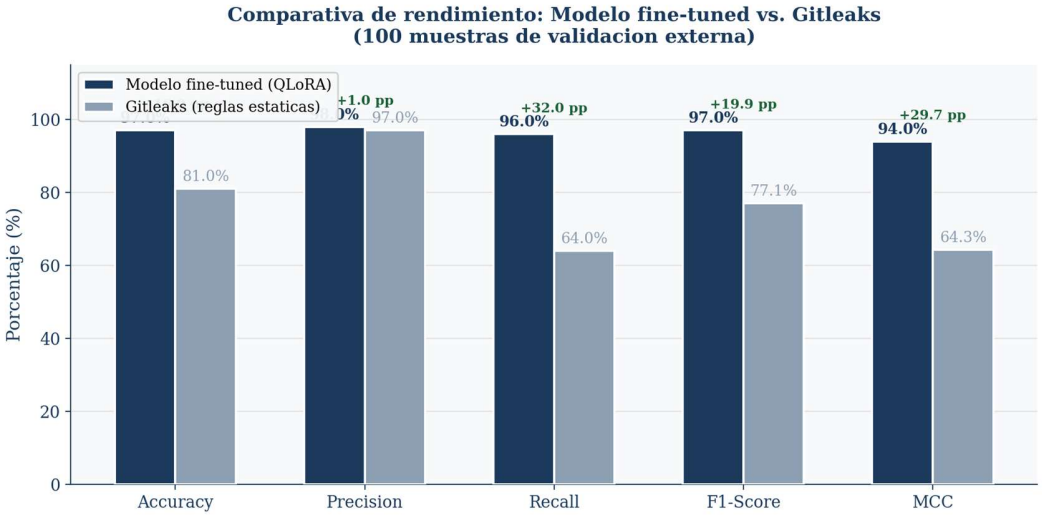

Figura 3. Comparativa de rendimiento entre el modelo fine-tuned y Gitleaks sobre las 100 muestras externas. El modelo supera a la herramienta de referencia en las cinco métricas, con ventajas especialmente marcadas en Recall (+32 pp) y F1-Score (+19,9 pp). Fuente: elaboración propia.

5.2. Interpretabilidad: Confianza Calibrada

La confianza media en aciertos es **1,000**, frente a **0,992** en errores: el modelo es ligeramente menos seguro cuando se equivoca, señal de calibración correcta. Sobre el Bloque A la confianza es 1,000 con accuracy del 100%. Sobre los edge cases, la confianza es 0,999 con accuracy del 85%. Los tres errores son: un fallback hardcodeado en os.getenv() (SAFE predicho como SECRET), un token marcado «fake» (SECRET predicho como SAFE) y un comentario con clave AKIA (SECRET predicho como SAFE). Los tres son genuinamente ambiguos.

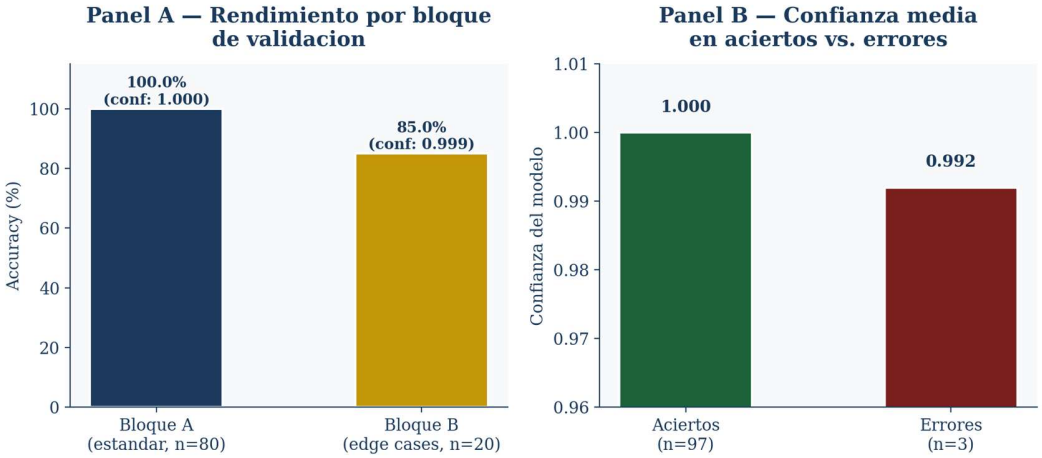

Figura 4. Panel A: rendimiento y confianza por bloque de validación. Panel B: confianza media en aciertos vs. errores, confirmando calibración correcta. Fuente: elaboración propia.

5.3. Generalización: Más Allá de la Clasificación

Tres pruebas demuestran que el modelo aprendió el concepto, no memorizó etiquetas. **Remediación automática**: propone versiones seguras con variables de entorno en los tres casos evaluados (Python y Java). **Explicación en lenguaje natural**: articula correctamente los riesgos de exposición en repositorios. **Clasificación multilenguaje**: 100% de aciertos (6/6) sobre JavaScript, Ruby, Go, PHP y YAML, lenguajes no representados en el entrenamiento. Solo el aprendizaje de una representación abstracta explica este resultado.

6. Conclusiones

6.1. Aprendizaje: El valor de la especialización sobre una base sólida

El desarrollo de este proyecto ha demostrado que no es necesario entrenar un modelo desde cero para obtener resultados precisos en ciberseguridad. La lección principal ha sido entender cómo usar un modelo fundacional robusto, en este caso, Qwen2.5-Coder. Este modelo base ya tiene una comprensión profunda de la sintaxis y estructura del software. Al aplicar el proceso de ajuste fino, no estamos enseñándole a programar, sino que estamos haciendo una especialización intensiva. Hemos tomado un sistema con un amplio conocimiento general y lo hemos centrado en la tarea crítica de identificar credenciales. Así, ha logrado distinguir el contexto de las vulnerabilidades sin perder su capacidad de razonamiento original.

6.2. Expectativa vs Realidad: Eficiencia en entornos limitados

Una de las grandes conclusiones de la investigación ha sido derribar el mito de que la aplicación de IA generativa avanzada requiere infraestructuras inalcanzables. La expectativa inicial sugería que trabajar con hardware de gama de entrada limitaría severamente la viabilidad del proyecto. Sin embargo, la realidad empírica demostró que, aplicando técnicas de optimización de memoria como la cuantización (NF4), es completamente viable ejecutar e instruir modelos de última generación en entornos con recursos computacionales restringidos (como una única GPU T4). Este hallazgo valida la metodología y demuestra que es posible democratizar las auditorías de código seguras en equipos con presupuestos limitados.

6.3. Retos de la Investigación y Amenazas a la Validez

Toda investigación empírica tiene limitaciones. Durante el desarrollo y la evaluación de este Trabajo Fin de Máster, se identificaron varios retos críticos que deben presentarse como amenazas a la validez del estudio:

• **Amenaza a la Validez Interna (Sesgo de Datos Sintéticos**): El mayor reto en la fase de entrenamiento fue la falta de repositorios públicos con secretos codificados reales y verificados, debido a las políticas de sanitización de plataformas como GitHub. Para abordarlo, se generaron plantillas sintéticas. Aunque se estableció un balance estricto y se aisló el conjunto de validación para evitar la fuga de datos, existe el riesgo de que el modelo haya ajustado demasiado su aprendizaje a los patrones de estas plantillas sintéticas. Esto podría causar un rendimiento

inferior ante código enmascarado o estructuras de inyección de credenciales poco comunes en entornos reales.

• **Amenaza a la Validez Externa (Generalización de Lenguajes)**: Aunque el modelo demostró una precisión excepcional, con un 100% de acierto empírico en el conjunto de validación, en cinco lenguajes de programación que no habían sido vistos previamente durante su ajuste fino, esta generalización semántica no garantiza el mismo rendimiento en lenguajes de programación de paradigmas muy diferentes, como los lenguajes funcionales puros o lenguajes de bajo nivel, como Ensamblador o C sin formatear.

• **Limitaciones de Precisión por Cuantización (Trade-off Técnico)**: La decisión metodológica de usar cuantización NF4 (4 bits) a través de QLoRA fue crucial para que el modelo pudiera ejecutarse en hardware de entrada, como la GPU T4. Sin embargo, este proceso implica una pérdida de información matemática en los pesos originales del modelo Qwen2.5-Coder. Aunque las métricas de MCC y los intervalos por bootstrap muestran que el impacto en la clasificación fue estadísticamente insignificante, representa una limitación frente a un ajuste fino completo con precisión de 16 o 32 bits.

• **Prevención de Fuga de Datos (Data Leakage):** En el contexto del aprendizaje automático aplicado a la ciberseguridad, un riesgo crítico es la posibilidad de que el modelo haya procesado los datos de prueba durante su entrenamiento, lo que invalidaría los resultados. Para mitigar esta amenaza a la validez interna, se implementó un riguroso protocolo de descontaminación cruzada durante la fase de preparación de datos. Se verificó exhaustivamente que ninguna variable, fragmento de código, ni estructura perteneciente al subconjunto de evaluación de *CodeSearchNet* estuviera presente en el corpus de entrenamiento. Esta estricta separación garantiza empíricamente que las métricas de rendimiento obtenidas reflejan una auténtica capacidad de generalización y comprensión semántica del modelo frente a código nunca visto, erradicando cualquier posibilidad de memorización.

Superar y gestionar estos retos no solo valida la solidez de la metodología empleada, sino que también sienta las bases para futuras líneas de investigación.

Agradecimientos

Mi reconocimiento más profundo a **Pablo Saucedo**, tutor de este TFM, cuyas preguntas transformaron una implementación técnica en un trabajo con rigor científico real. Al claustro del máster: **Pablo González** (Telefónica), **Fernando Wanguemert** (TU Berlín/TU Delft), **Pepe Ballesteros** (Max Planck), **Francisco Ramírez** (DefCon/BlackHat), **David García** (Telefónica Tech), **Rafael Troncoso** (Tesla) y **José Torres** (ElevenPaths). Y, por último, pero no por ello son menos importantes: a mi familia, a mi empresa, y a mis amigos, por aceptar que «estoy con el Máster o con el TFM» fuera respuesta válida a cualquier oferta de plan.

Bibliografía

[1] GitGuardian, "State of Secrets Sprawl 2025," 2025. [En línea]. Disponible en: https://www.gitguardian.com/state-of-secrets-sprawl.

[2] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," en Proc. ICLR, 2022.

[3] Qwen Team, Alibaba Cloud, "Qwen2.5-Coder Technical Report," arXiv preprint arXiv:2409.12186, 2024.

[4] R. Saha et al., "HasSecret: Detecting Hardcoded Secrets in Source Code," en Proc. IEEE SANER, 2024.

[5] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," en Proc. NeurIPS, 2023.

[6] Samsung Research, "CredData: A Dataset of Credentials for Research," 2021. [En línea]. Disponible en: https://github.com/Samsung/CredData.

[7] H. Husain et al., "CodeSearchNet Challenge," arXiv preprint arXiv:1909.09436, 2019.

[8] T. Han et al., "Parameter-Efficient Fine-Tuning for Large Models," arXiv preprint arXiv:2403.14608, 2024.

[9] Y. Ding et al., "A Survey on Vulnerability Detection with Large Language Models," arXiv preprint arXiv:2401.17170, 2024.

[10] X. Chen et al., "Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities," IEEE Transactions on Software Engineering, 2024.

[11] M. Jimenez et al., "Vulnerability Detection in the Era of LLMs: Challenges and Opportunities," en Proc. International Conference on Software Engineering (ICSE), 2024.

[12] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications v2.0," 2025. [En línea]. Disponible en: https://genai.owasp.org/.

[13] L. Zheng et al., "LlamaFactory: Unified Efficient Fine-Tuning of 100+ LLMs," arXiv preprint arXiv:2403.13372, 2024.

[14] D. Guo et al., "DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence," arXiv preprint arXiv:2401.14196, 2024.

[15] Y. Wang et al., "A Survey on Quantization Methods for Large Language Models," arXiv preprint arXiv:2402.14620, 2024.

Anexos

Anexo A: Prueba de Línea Base

Antes de aplicar LoRA, el modelo base no responde al formato de clasificación: genera texto libre, código adicional o explicaciones en prosa. Esta prueba demuestra que la mejora es atribuible al fine-tuning.

Fragmento	Correcta	Respuesta del modelo base
AWS_SECRET = 'AKIAIOSFODNN7EXAMPLE'	SECRET	Genera explicación en prosa
timeout_seconds = 60	SAFE	Respuesta inconsistente
password = 'p@ssw0rd123!'	SECRET	Genera código de validación
GITHUB_TOKEN = 'ghp_xxxxxxxxxxxx'	SECRET	Texto libre sobre seguridad
db_url = os.getenv('DATABASE_URL')	SAFE	Explica que es seguro, sin etiqueta

Anexo B: Ficha Técnica del Experimento

Parámetro	Valor
Modelo Base	Qwen2.5-Coder-7B (NF4, 4 bits)
Infraestructura	Google Colab, GPU NVIDIA T4 16 GB VRAM (gratuito)
Técnica	QLoRA: LoRA r=16, α=16 + Cuantización NF4
Módulos LoRA	q, k, v, o, gate, up, down (7 módulos)
Dataset	560 muestras: 140 CredData + 130 CSN + 290 sintéticas
Batch Efectivo	8 (batch=2 × gradient accumulation=4)
Learning Rate	2×10^{-4} con scheduler coseno
Pasos	200 + 20 de calentamiento
Duración	12,7 minutos
Validación Externa	100 muestras: 80 estándar + 20 edge cases
Accuracy (total)	97,0% [IC 95%: 94%-100%]
F1-Score (total)	97,0% [IC 95%: 92,9%-100%]
Bloque A estándar	100% en todas las métricas
Multilenguaje	6/6 lenguajes no vistos (100%)
Parámetros entrenados	40,4M de 4.393M (0,92%)